genomDE2FHIR: Stand, offene Punkte und Leitfaden zum Code

16. Oktober 2026

Repository: https://github.com/KohlbacherLab/genomDE2FHIR, Release **v0.11.0** (10.10.2026). Alle Pfade unten sind relativ zum Repository.

1. Worum es geht

Wir bilden die drei Datenkranz-Zweige des Modellvorhabens Genomsequenzierung (KDK Onkologie, KDK Seltene Erkrankungen, GRZ) auf den **MII KDS 2027.0.0-Ballot** ab (FHIR R4; das Profilset 2026 bleibt wählbar). Es gibt drei Teile:

- **(a) `genomde2fhir`**, ein Python-Konverter: JSON → Transaction-Bundle, deterministische IDs, `PUT`, idempotent.
- **(b) FML-StructureMaps**, die dasselbe in matchbox per $transform leisten.
- **(c) Validierung mit matchbox**: Jedes Problem wird klassifiziert (eigener Fehler, Quelldaten, nicht prüfbare Terminologie, Paketdefekt).

Die **Mapping-Tabelle** (`mapping/mapping_*.csv`, eine Zeile je Schema-Blatt) ist die verbindliche Referenz; beide Implementierungen folgen ihr.

Stand. (a) und (b) erzeugen auf allen 1.259 schemagültigen synthetischen Dateien identische Bundles. Alle 1.025 ungültigen Dateien werden abgewiesen. Die Bundles laden zweimal fehlerfrei in HAPI. In den validierten Stichproben erzeugt unser Konverter keine eigenen Konformitätsfehler. Die verbleibenden Validierungsfehler stammen aus den Eingangsdaten, aus Terminologien, die tx.fhir.org nicht prüfen kann (BfArM-Systeme, ICD-O-3), oder aus Defekten der Ballot-Pakete.

2. Offene Punkte

Die offenen Punkte stehen in drei Dokumenten mit je eigenem Zweck:

Dokument	Inhalt	Von Ihnen benötigt
`docs/QUESTIONS-TD-2026-10-10.md` (+ PDF)	25 Rückfragen zu Ihrer annotierten Tabelle vom 06.10. (205 Zeilen geprüft, 140 übernommen)	Antworten
`docs/BALLOT-COMMENTS-2027.md`	44 Entwürfe für Ballot-Kommentare (ONKO-1…9, MTB-1…23, MOLGEN-1…7, SE-1…3, CONSENT-1, BIOBANK-1…3); Übersicht am Ende	Wer reicht ein, Frist 27.10.
`docs/REPRESENTATIONS.md`	jede lokale Interimslösung (Extension, CodeSystem, Profil) mit Begründung, nummeriert `#n`	Prüfung

Jede Rückfrage in `QUESTIONS-TD` ist gleich aufgebaut: die Frage, *Ihre Tabelle* (Zeile und Ihr Vorschlag) und *Wir derzeit* (was der Code tut und warum). Die wichtigsten Punkte, mit Fundstellen im Code:

#	Thema	Problem	Unsere Interimslösung	Python	FML
1	MV-Einwilligung	Für mvSequencing, reIdentification und caseIdentification gibt es keine Policy-Codes. `provision.code` ist required gebunden, die OIDs .2/.6/.8 bedeuten Broad-Consent-Zwecke.	Lokales Profil `mvgs-consent` mit lokalen Codes, bewusst nicht konform zu `einwilligung` (CONSENT-1)	`builders/consent.py`	`DkMvConsent` (Common)
2	HGVS	Unqualifizierte `c.`/`p.`-Zeichenketten scheitern an der required-Bindung an `hgvs-vs`.	Mit Transkript qualifiziert, die Rohzeichenkette bleibt in `text`	`builders/onco/molecular_variants.py`	`OncoMolHgvsText`
3	Transkript-Bindung	`representative-transcript-ref-seq` ist required an `http://ensembl.org` gebunden (kein ValueSet) und blockiert RefSeq.	System nach Präfix (ENST/NM_) (MTB-2)	`builders/onco/molecular_variants.py`	`OncoMolTranscriptCoding`
4	TNM 2027	`coding:uicc` ist 1..1, der KDK liefert nur SNOMED. Die MII-Maps UICC→SNOMED sind unvollständig.	Lokale Umkehr-Map; nicht zuordenbare Einträge gehen nach `weitere-klassifikationen` (ONKO-5)	`builders/onco/tnm.py`	`OncoTnm*`
5	ECOG	`5` und `notApplicable` fehlen im CodeSystem; `coding:obds` ist 1..1.	Keine Observation, als verworfen gemeldet (ONKO-4)	`builders/onco/ecog.py`	`OncoEcog*`
6	Response	`therapyResponseSource` ist eine Datenquelle, keine RECIST/RANO-Methode; `method` ist 1..1.	Lokales Response-Profil, Quelle in `note` (MTB-12)	`builders/onco/followup.py`	`OncoFuResponse*`
7	Studien-Evidenz	Die Evidenzgraduierung ist an ServiceRequest nicht zulässig.	Lokale Extension `mvgs-ex-evidence-level` mit den MII-CodeSystemen (MTB-13)	`builders/onco/careplan.py`	`OncoPlanEvidence*`
8	ACMG-Kriterien (SE)	Kein Element in MolGen, MTB oder GR.	Ungeslicte Komponente an `diagnostische-implikation` (MOLGEN-6)	`builders/rd/variants.py`	`RdAcmgStrength`
9	Segregation, Erbgang	value[x] ist 1..1 und erlaubt kein `notPerformed` oder `unclear`.	Komponente weglassen bzw. nur Text (MOLGEN-5)	`builders/rd/variants.py`	`RdVariante`, `RdImplication*`
10	GRZ-QC	Die `qc`-Extension ist im Kontext Extension statt Procedure deklariert; `minCoverage` ≠ `sequencing-coverage`.	`metrics-description`; Fehler als `waived` (MTB-3, MTB-19)	`builders/grz/labdata.py`	`GrzQc*`

Die Punkte 11–25 (Metachronie, weitere Klassifikationen, Vorbefunde, Expression, LOH, COSMIC usw.) stehen vollständig in `QUESTIONS-TD`.

Vor dem 27.10. zu klären: (1) Wer reicht die Ballot-Kommentare ein, Sie oder wir, und in welcher Form (HDB-Ticket je Kommentar oder je Modul)? (2) Abgleich mit den Texten von HDB-975/976 (SE-1, SE-2, MOLGEN-3…5). (3) Bestätigung der Frist.

Weitere offene Punkte auf unserer Seite (`docs/PLAN-2027.md` §11): Unser Eingangsvalidator (genomde-dk-validator 1.1.2)

kennt nur Consent-`schemaVersion` 2025.0.1. Die KDK- und GRZ-Schemata erlauben seit 08/2026 auch 2025.0.2–2026.0.1; Release 1.1.3 ist in Arbeit. Der KDK-Zweig `develop` (unveröffentlicht, `confirmationId`, Feldumbenennung `intervention IsRiskReducing`) ist noch nicht eingearbeitet. Das synthetische Korpus `example-data/synthData-v1` ist wegen ungeklärter Herkunft nicht im Repository. Für den GRZ-Zweig fehlen Testdaten mit befüllter Broad-Consent-`scope`.

3. Leitfaden zum Code

3.1 Ausprobieren

```
pipx install ./genomde2fhir
pipx inject genomde2fhir \
  "genomde-dk-validator @ git+https://github.com/KohlbacherLab/genomde-dk-validator@v1.1.2"
genomde2fhir convert examples/oncology/ -o out/ --report report.json
genomde2fhir validate out/*.bundle.json --matchbox <matchbox-URL>
```

`report.json` zeigt je Datei, welche Felder abgebildet (*covered*), unverändert erhalten (*preserved*) oder mit Begründung **verworfen** (*dropped*) wurden. Damit lässt sich am schnellsten prüfen, was mit einem bestimmten Feld passiert. Ein matchbox mit den 2027-Paketen: `infra/matchbox-2027/`.

3.2 Python-Konverter (`genomde2fhir/src/genomde2fhir/`)

Empfohlene Lesereihenfolge:

1. **`convert.py`** – Einstieg `convert()`: Eingang prüfen, Zweig erkennen, Builder-Pipeline laufen lassen, Bundle und Provenance bauen.
2. **`builders/__init__.py`** – `PIPELINES`: die Reihenfolge der Builder je Zweig. Der Docstring listet die gemeinsamen Referenznamen (z. B. *primary* für den Primärtumor), über die spätere Builder auf frühere Ressourcen verweisen.
3. **`context.py`** – `ctx`: `ctx.new()` erzeugt eine Ressource mit Profil und deterministischer ID, `ctx.add()` registriert sie. `ctx.drop()`, `ctx.warn()` und `ctx.preserve()` füllen den Bericht.
4. **`builders/onco/`, `builders/rd/`, `builders/grz/`** – ein Modul je Datenkranz-Block, gemeinsame wie `patient.py`, `consent.py`, `research_consent.py`). **Jeder Modul-Docstring beginnt mit `Rows:`**, den Tabellenzeilen, die er umsetzt, gefolgt von den Zielprofilen und den Abweichungen mit Verweis auf REPRESENTATIONS #n.
5. **`identity.py`**, **`bundle.py`** – Ressourcen-IDs (`uuid5` aus Sender, TAN und lokalen IDs), `meta.source` und Bundle-Einträge; die Regeln stehen in `docs/IDENTITY.md`.
6. **`validate.py`** – Aufruf von matchbox und Klassifikation jedes Problems; `WAIVERS` listet die bekannten Paketdefekte, jeder mit Verweis auf einen Ballot-Kommentar.

Von einer Tabellenzeile zum Code: Den Feldnamen aus der Spalte *path* in `mapping/mapping_*.csv` suchen (z. B. *molecular.smallVariants[].transcriptId*); der Treffer ist der Builder, dessen `Rows:` den Pfad nennt:

```
grep -rn "transcriptId" genomde2fhir/src/genomde2fhir/builders mappers/fml
```

3.3 FML-Maps (`mappers/fml/`)

Map	Inhalt
`DatenkranzCommon.map`	gemeinsame Gruppen `Dk*`: Patient, Coverage, MV-Consent, Broad-Consent-Migration, Datentypen
`DatenkranzOncologyToMii.map`	Onkologie, Hauptmap (`Onco*`); importiert die beiden folgenden
`OncoMolecular.map`	molekularer Block (`OncoMol*`)
`OncoPlanFollowUp.map`	Therapieplan und Follow-up (`OncoPlan*`, `OncoFu*`)
`DatenkranzRareDiseaseToMii.map`	Seltene Erkrankungen (`Rd*`)
`DatenkranzGrzToMii.map`	GRZ (`Grz*`)

Quelltypen sind generierte logische Modelle der drei Zweige (`mvgs-ig/input/StructureDefinition-Datenkranz*.json`). Die Gruppennamen folgen den Python-Modulen; zur Tabelle oben gehören z. B. `OncoTnm*` ↔ `onco/tnm.py` und `GrzQc*` ↔ `grz/labdata.py`. `mappers/fml/README.md` beschreibt das Laden in matchbox (Skript `scripts/build_mvgs.py` mit den

Optionen *install* und *maps*) und die geprüften Eigenheiten der FML-Engine (mit Testmaps in `test/`). Bewusste Vereinfachungen gegenüber Python sind dort aufgeführt.

3.4 Lokale Artefakte und Dokumentation

- `mvgs-ig/` – Paket `de.genomde.mvgs`: logische Modelle sowie lokale Profile, Extensions, CodeSystems und ConceptMaps (z. B. Lokalisation → SO, TNM SNOMED → UICC), nur dort, wo der Ballot kein konformes Element hat.
- `docs/REPRESENTATIONS.md` – Register dieser Interimslösungen (`#n`), auf das Code und Tabelle verweisen.
- `docs/E2E-REPORT-2027.md` – Zahlen des letzten Gesamtlaufs je Zweig.
- `docs/PLAN-2027.md` **§11** – Status und alle offenen Punkte.
- **Tests:** `genomde2fhir/tests/` (1.205 Tests, davon ein Teil gegen matchbox/HAPI).